Traffic Sign Recognition for Autonomous Vehicles

Authors:

Ahkar Htet
Cassio Santanna
Bernardo Xavier
Eduar Stiven

Institution: Fanshawe College
Program: AIM1 – Artificial Intelligence and Machine Learning (Co-Op)
Course: INFO-6152 Deep Learning With Tensorflow & Keras 2

Rev.	Rev. Description	Issued by	Issue date
0	Initial issue.	Group 8	2025-04-17

TABLE OF CONTENTS

1.0 INTRODUCTION

Autonomous driving systems depend on recognizing the surroundings. Traffic-Sign Recognition (TSR) plays a fundamental role in these systems, enabling the vehicles interpret and make decisions to road signs in real time.

Deep Learning models has played a significant role in this field. In this project, we are going to explore two approaches: Convolutional Neural Networks (CNNs) and YOLO (You Only Look Once) models.

2.0 PROBLEM STATEMENT

The goal of this project is to develop a TSR system capable of classifying and detecting traffic signs. The key challenges include:

- Handling variations in lighting, weather, and sign occlusion.
- Managing class imbalance in the dataset (some signs appear more frequently than others).
- Achieving high accuracy while maintaining real-time performance for autonomous driving applications.

We address these challenges using two approaches:

- CNN-based classification for recognizing traffic signs from cropped images.
- YOLO-based object detection for locating and classifying signs in full driving scenes.

3.0 DATASET SELECTION

We used the **German Traffic Sign Recognition Benchmark (GTSRB)** dataset from Kaggle, which contains:

- **43 classes** of traffic signs.
- **>50,000 images** in total (training + test sets).
- High-resolution images (varying sizes) with annotations.
- Key features:
    - Real-world variability (illumination, occlusions, rotations).
    - Annotations include class labels and bounding box coordinates (for detection tasks).

The dataset is a standard benchmark in TSR research due to its widely adoption and reliability. It features a diverse set of classes, including regulatory, warning, and informational signs. Ensures coverage of real-world traffic scenarios. Additionally, its large size provides enough data for training deep learning models effectively, making it a valuable resource for developing and evaluating computer vision systems in autonomous driving systems. The Figure 1 presents samples of the dataset.

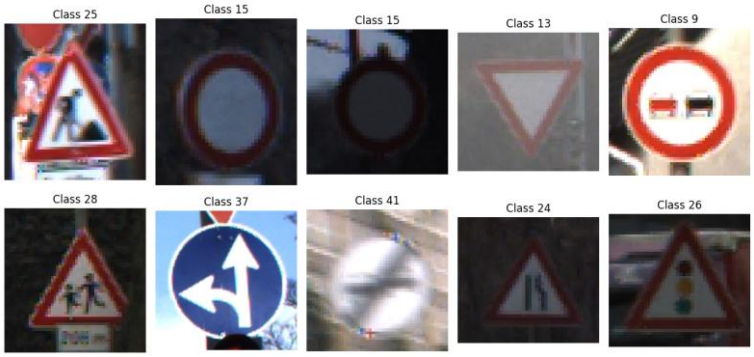


Figure 1: Samples of the dataset

3.1 Data Exploration

3.1.1 Class Distribution

First, we verified if the dataset was balanced, to avoid the risk of underfitting in minority classes and to verify the necessity of balancing the dataset. The Figure 2 shows that the dataset is not well balanced.

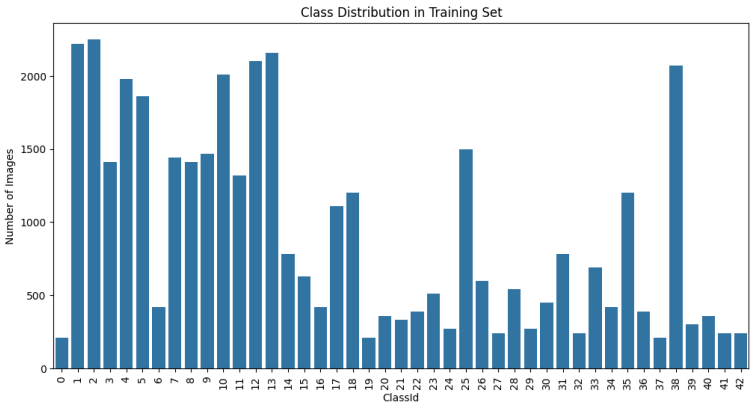


Figure 2: Class distribution

3.1.2 Image size distribution in metadata

Most of the pitures are 50x50 or smaller, as presented in Figure 3.

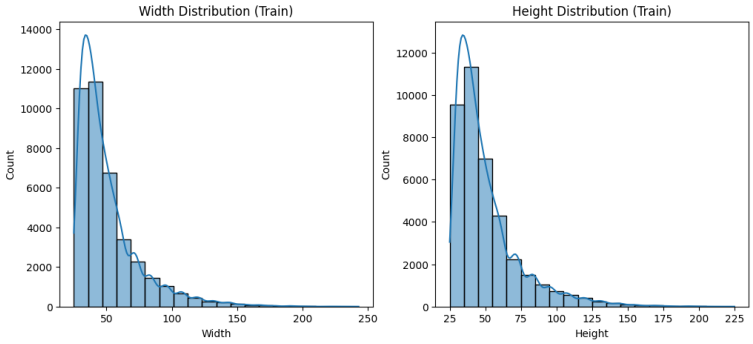


Figure 3: Image size distribution in metadata

3.1.3 Image Quality Analysis

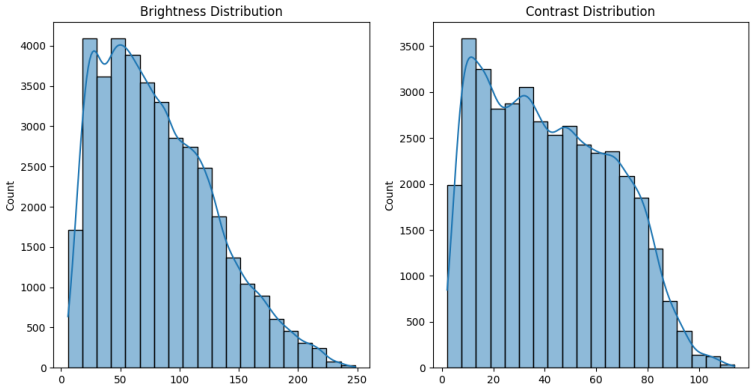


Figure 4: Image size distribution in metadata

3.1.4 Correlation between variables

We checked for correlations between numerical columns (such as `Width, Height, Roi.X1, Roi.Y1, etc.`) using a ‘correlation matrix’, presented in the

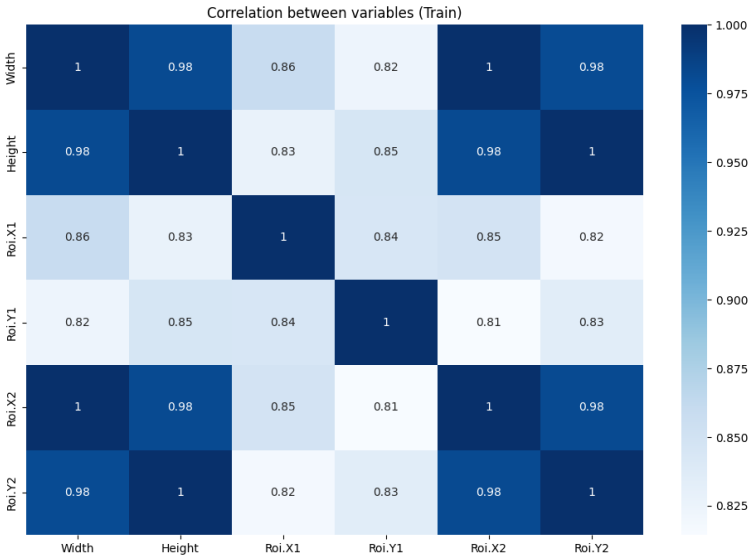
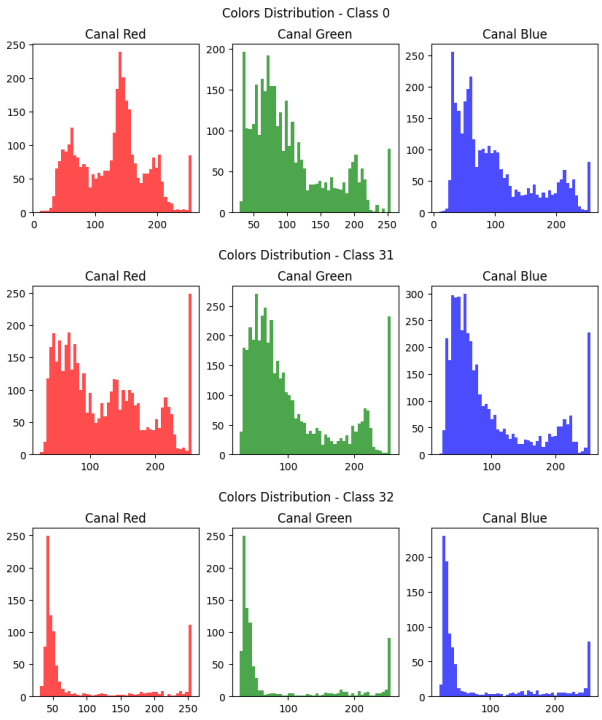


Figure 5: Correlation between variables

3.1.5 Color and Intensity Analysis

Since the images are of traffic signs, pixel colors and intensity can be informative:

- **RGB Channel Distribution**: Selected a few random images and analyze the distribution of 'RGB channel' values to understand the dominant colors in the dataset.



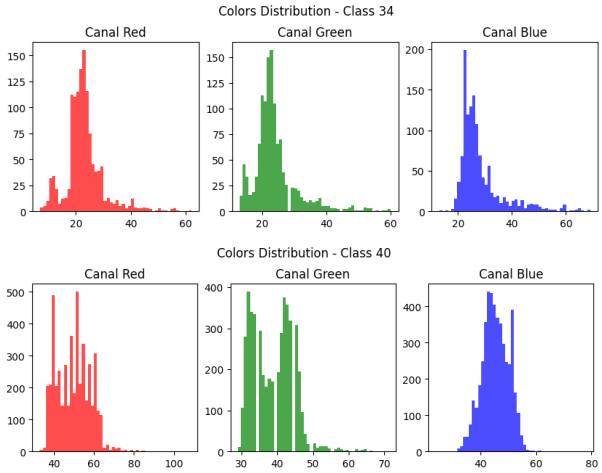


Figure 6: Color and Intensity Analysis, classes 0, 31, 32, 34 and 40

4.0 DATA PREPARATION

We performed the following steps to preparing the data to train the models:

1. **Loading Data:**
    i. Images were loaded from directories and resized to **32x32 pixels** (for CNN) and **416x416** (for YOLO).
    ii. Metadata (class labels, bounding boxes) was parsed from CSV files.

2. **Train-Validation-Test Split:**
    i. **80% training**, **20% validation** (stratified to maintain class balance).
    ii. Separate test set provided by GTSRB.

3. **Preprocessing:**
    i. Normalization: Pixel values scaled to **[0, 1]**.
    ii. Label Encoding: Converted to one-hot vectors for classification.

4. **Handling Corrupted Images:**
    i. Verified image integrity using PIL.Image.verify().
    ii. No corrupted images were found in the dataset.

5.0 DATA AUGMENTATION

To improve model generalization, we applied the following transformations during training:

- **Rotation (±30 degrees)**
- **Width/Height Shifting (±20%)**

- **Zooming (±10%)**
- **Fill Mode:** Nearest-neighbor interpolation for empty regions.

The Figure 7 presents samples of augmented images

Figure 7: Samples of augmented images

6.0 CNN MODEL DEVELOPMENT

We designed a **3-layer CNN** with the following structure:

1. Conv2D (32 filters, 3x3 kernel) → BatchNorm → MaxPooling
2. Conv2D (64 filters, 3x3 kernel) → BatchNorm → MaxPooling
3. Conv2D (128 filters, 3x3 kernel) → BatchNorm → MaxPooling
4. Flatten → Dense (256 units) → Dropout (0.5) → Softmax

Number of parameters:

- Total params: 236,523 (923.92 KB)
- Trainable params: 236,075 (922.17 KB)
- Non-trainable params: 448 (1.75 KB)

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
batch_normalization (BatchNormalization)	(None, 30, 30, 32)	128
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 13, 13, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 4, 4, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 2, 2, 128)	0
flatten (Flatten)	(None, 512)	0
dense (Dense)	(None, 256)	131,328
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 43)	11,051

Figure 8: Model parameters

Training Configuration

- **Optimizer:** Adam (learning rate = 0.001)
- **Loss:** Categorical Cross-Entropy
- **Batch Size:** 32
- **Epochs:** 20
- **Callbacks:** Early stopping (patience=3), ReduceLROnPlateau.

7.0 CNN MODEL RESULTS

The model demonstrates effective learning, with training and validation loss decreasing greatly in the early epochs. Validation accuracy quickly stabilizes around 99%, indicating strong generalization. The validation loss remains low and stable, with no signs of overfitting, suggesting the model fits the training data well and achieves high accuracy with minimal overfitting. The Figure 9 presents the training curves.

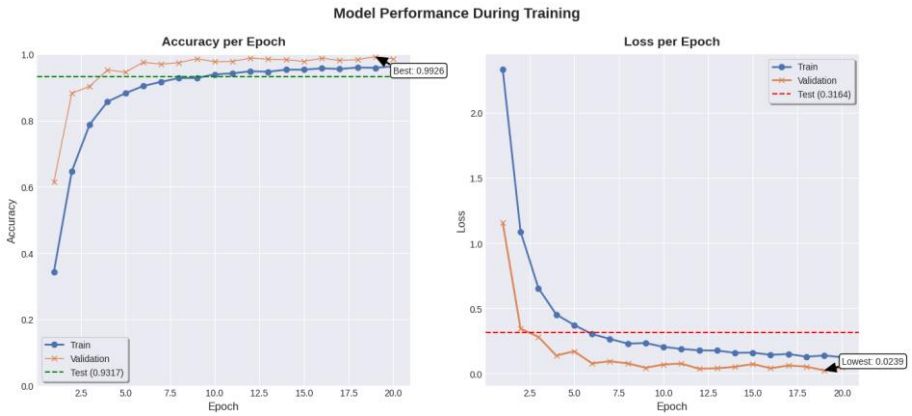


Figure 9: CNN training curves

7.1 Confusion Matrix

The high values in the diagonal of the Confusion Matrix indicates correct classifications. Some misclassifications occurred for visually similar signs or bad quality images. The Figure 10 presents the CNN Confusion Matrix.

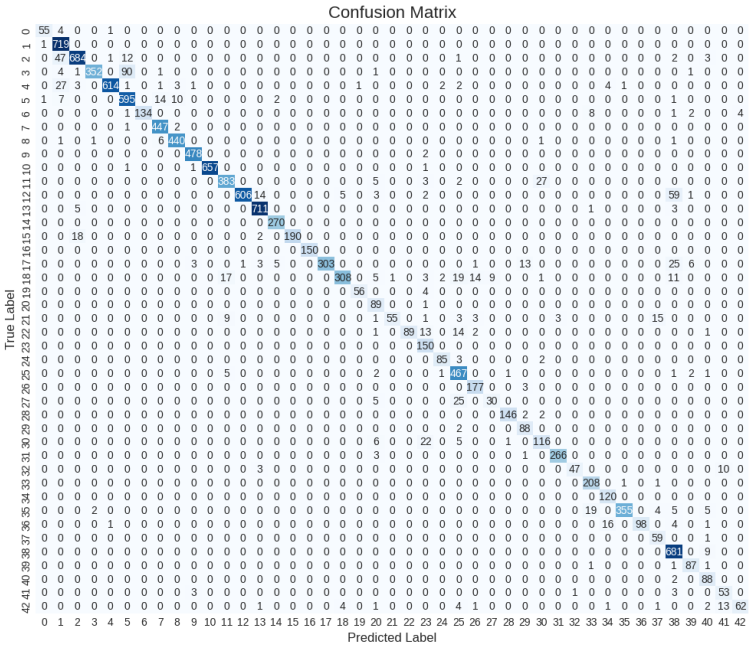


Figure 10: CNN Confusion Matrix

8.0 CNN TESTING

As expected by the training results, the model was able to predict correctly in a high rate, as presented in the Figure 11.

Figure 11: Predictions of the CNN Model

9.0 YOLO MODEL DEVELOPMENT

In the second part of the project, we shifted from image classification to object detection using the YOLOv8 model to identify and localize traffic signs directly within driving scenes. Unlike CNNs which work on cropped images, YOLO allows real-time full-frame detection.

YOLOv8 Configuration:

- **Model**: YOLOv8n (nano) for fast and lightweight detection.
- **Input Size**: 640×640 (adjustable to fit GPU resources and sign sizes).
- **Pretrained Weights**: COCO dataset.
- **Optimizer**: SGD with warm-up and cosine decay.
- **Training Epochs**: 15
- **Loss Functions**: Box loss, classification loss, objectness loss (as defined by Ultralytics YOLOv8 framework).

Data Preparation Pipeline:

- Raw data from Train.csv, Test.csv, and Meta.csv was merged.
- Manual label mapping was applied to associate ClassId with human-readable traffic sign names.
- To ensure dataset balance, only classes with at least **500 samples** were retained, and each was limited to exactly **500 images**.
- Bounding boxes were converted into **YOLO format** (x_center, y_center, width, height) and saved into .txt annotation files.
- Data was split into **Train (70%)**, **Validation (15%)**, and **Test (15%)**, and organized into folders according to YOLO directory conventions.

10.0 YOLO MODEL RESULTS

Training Performance:

The YOLOv8 model demonstrated strong convergence:

- **Precision**: >0.99
- **Recall**: >0.98
- **mAP@0.5**: ~0.995
- **mAP@0.5:0.95**: ~0.93

Loss graphs show clear downward trends across training and validation sets. There was no overfitting, and the validation losses remained consistently low, indicating stable learning.

```
Model summary (fused): 72 layers, 3,010,523 parameters, 0 gradients, 8.1 GFLOPs
val: Scanning /content/yolo_dataset/val/labels.cache... 1500 images, 0 backgrounds, 0 corrupt: 100%|██████████| 1500/1500
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 47/47
                   all       1500       1500      0.997      0.995      0.995      0.955
           Speed limit 30         60         60      0.998      0.983      0.995      0.956
           Speed limit 50         60         60      0.996          1      0.995      0.949
           Speed limit 60         60         60      0.996      0.967      0.994      0.948
           Speed limit 70         60         60          1       0.97      0.995      0.955
           Speed limit 80         60         60      0.966          1      0.994      0.943
          Speed limit 100         60         60      0.998          1      0.995      0.949
          Speed limit 120         60         60      0.991      0.983      0.995      0.926
             No passing           60         60      0.998          1      0.995      0.968
No passing for vehicles over 3.5 tons         60         60      0.999          1      0.995      0.968
Right of way at intersection         60         60      0.999          1      0.995      0.963
           Priority road         60         60      0.999          1      0.995      0.961
                   Yield         60         60          1      0.993      0.995      0.972
                    Stop         60         60      0.999          1      0.995      0.964
            No vehicles          60         60          1       0.99      0.995      0.947
               No entry          60         60      0.999          1      0.995      0.936
         General caution         60         60      0.999          1      0.995      0.946
          Slippery road          60         60      0.998          1      0.995      0.974
              Road work          60         60      0.999          1      0.995       0.98
         Traffic signals         60         60      0.999          1      0.995      0.945
      Children crossing          60         60      0.999          1      0.995      0.974
     Beware of ice/snow          60         60      0.998          1      0.995      0.954
  Wild animals crossing          60         60      0.999          1      0.995      0.965
      Turn right ahead           60         60      0.999          1      0.995      0.951
             Ahead only          60         60      0.999          1      0.995      0.939
             Keep right          60         60      0.999          1      0.995      0.931
Speed: 0.6ms preprocess, 9.1ms inference, 0.0ms loss, 0.9ms postprocess per image
Results saved to runs/detect/custom_yolo_train22
```

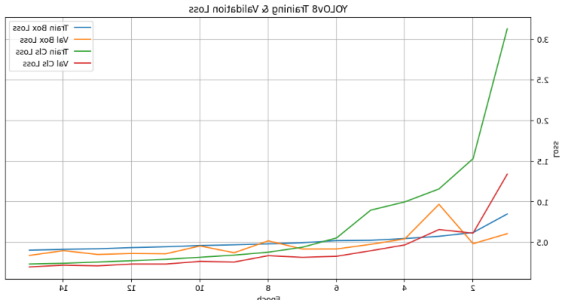


Figure: *YOLOv8 Class-wise Performance Summary*

11.0 YOLO TESTING

The trained YOLOv8 model was tested on a **realistic dashcam-style driving video**. It was able to:

- Detect and label traffic signs such as **"Stop"**, **"No Entry"**, and **"Speed Limit 30"** in real time.
- Produce accurate bounding boxes with confidence scores above 0.90 in most frames.

However, the model occasionally showed false positives or enlarged boxes. This is likely due to:

- Lack of high-resolution training data for certain signs.
- Limited GPU memory, leading to smaller batch sizes and lower image quality during training.

Note: Reducing image size from 640×640 to 256×256 may improve speed but risks lowering detection accuracy for smaller signs.

12.0 CONCLUSION

Convolutional Neural Networks (CNNs) are highly effective at image classification but lack the ability to precisely localize objects within an image. This limitation makes them less suitable for tasks that require object detection and spatial awareness.

On the other hand, YOLO offers real-time object detection with high speed, though it may sacrifice some accuracy compared to traditional methods.

This project demonstrated that when combined, CNN and YOLO create a powerful and balanced pipeline ideal for autonomous driving, leveraging CNN's classification strength and YOLO's localization and speed.

13.0 NEXT STEPS

- Improve the performance of classes that has less samples using focal loss or oversampling.
- Deploy on edge devices (NVIDIA Jetson) for real-world testing.
- Explore transformer-based models (e.g., ViT) for hybrid approaches.

14.0 REFERENCES

[1] Kaggle. (2011). *German Traffic Sign Recognition Benchmark (GTSRB) dataset*. Kaggle. https://www.kaggle.com/datasets/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign

[2] TensorFlow. (2023). *Image classification with CNNs*. TensorFlow Documentation. https://www.tensorflow.org/tutorials/images/cnn

[3] Ultralytics. (2023). *YOLOv5: Custom object detection training*. GitHub. https://github.com/ultralytics/yolov5

[4] OpenCV. (2023). *Image preprocessing techniques*. OpenCV Documentation. https://docs.opencv.org/4.x/d9/df8/tutorial_root.html

[5] Roboflow. (2023). *What is Data Augmentation? The Ultimate Guide*. Roboflow Blog. https://blog.roboflow.com/data-augmentation/

[6] Scikit-learn. (2023). *Confusion matrix visualization*. scikit-learn Documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html

[7] Papers With Code. (2023). *State-of-the-art traffic sign detection models*. https://paperswithcode.com/task/traffic-sign-recognition